

INFORMATION STORAGE AND MANAGEMENT

UNIT II – Data Protection (RAID) and Intelligent Storage Systems

Structured Exam-Ready Notes

SYLLABUS (from Unit-II PPT): Data Protection, Intelligent Storage System: Implementation of RAID, RAID Array Components, RAID Levels, RAID Comparison, RAID Impact on Disk Performance, Hot Spares, Components of an Intelligent Storage System, Intelligent Storage Array, Object-Based Storage and Unified Storage Platform.

HOW TO READ: PPT content is kept in its original wording. Topics in the syllabus that the PPT does not explain are marked with a red **SYLLABUS ADDITION** box. Blue = Definition, Green = Formula, Yellow = Key Point, Purple = Mini Summary. Notes on the PPT's own errors are marked **CHECK**.

NOTE: The syllabus text you pasted in your message is the Unit-I syllabus (already covered in the Unit-I notes). These notes follow the Unit-II syllabus given on slide 1 of this PPT.

Contents

1. Data Protection: RAID – Introduction and Why RAID?
2. Implementation of RAID, RAID Array Components and RAID Techniques
3. RAID Levels (0, 1, Nested 0+1 / 1+0, 2, 3, 4, 5, 6)
4. RAID Comparison
5. RAID Impact on Disk Performance
6. Hot Spares
7. RAID and Flash Drives; Conclusion
8. Intelligent Storage System: Introduction and Components (Front End, Cache, Back End, Physical Disks)
9. Logical Unit Number (LUN) and LUN Masking
10. Intelligent Storage Array (High-end and Midrange)
11. EMC Symmetrix Storage Array
12. Object-Based Storage and Unified Storage Platform
13. Benefits of Intelligent Storage Systems
14. Solved Problems and Review Questions
15. Quick Revision

1. Data Protection: RAID

DEFINITION: A **RAID** is a **Redundant Array of Inexpensive / Independent Disks**.

1.1 Introduction

- “Cheap” SCSI hard drives are now big enough for most applications.
- We use RAID today for:
 - **Increasing disk throughput** by allowing parallel access
 - **Eliminating the need to make disk backups** – disks are too big to be backed up in an efficient fashion

1.2 Disk Arrays

- Disk arrays with **redundant disks to tolerate faults**.
- If a single disk fails, the lost information is **reconstructed from redundant information**.
- **Striping:** simply spreading data over multiple disks.
- **RAID:** redundant array of inexpensive/independent disks.

1.3 Why RAID?

- Performance limitation of disk drive.
- An individual drive has a certain life expectancy, measured in **MTBF (Mean Time Between Failure)**.
- The more the number of HDDs in a storage array, the larger the probability for disk failure.

FORMULA: MTBF of array = MTBF of one drive / number of drives

Example: drive MTBF = 750,000 hours, 100 drives → array MTBF = $750,000 / 100 = 7,500$ hours

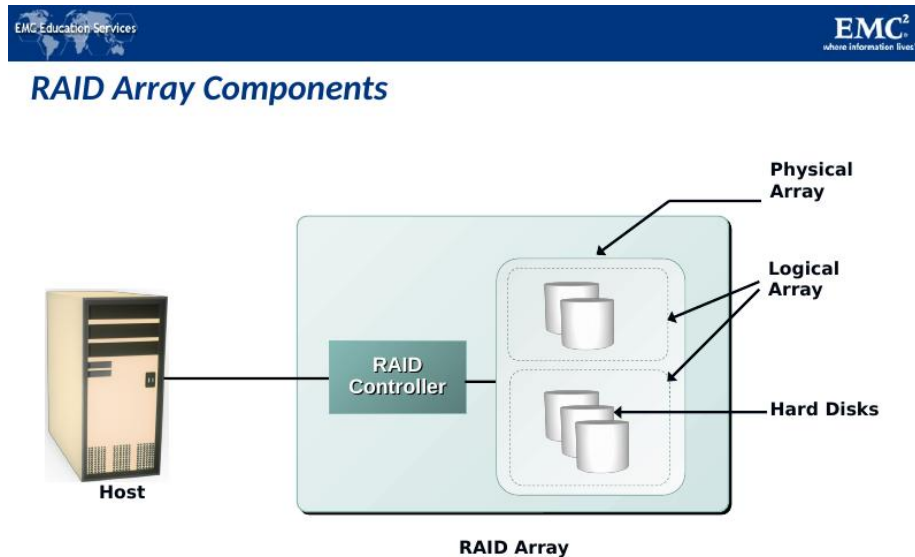
- RAID was introduced to mitigate this problem. **RAID provides:** Increased capacity, Higher availability, Increased performance.

KEY POINT: Business-critical applications need high performance, availability, security and scalability. A single HDD (mechanical) limits performance; RAID improved performance and reliability, and the **intelligent storage system** (Part B of this unit) evolved as a feature-rich RAID array with optimized I/O processing.

2. Implementation of RAID and RAID Array Components

2.1 RAID Array Components

A **RAID array** consists of a **RAID controller**, **hard disks (physical array)** and the **logical array** presented to the host.



RAID array = RAID controller + hard disks (physical array) + logical array (PPT Slide 50)

2.2 RAID Controller – Hardware vs Software Implementation

Hardware RAID	Software RAID
Usually a specialized disk controller card	Runs as part of the operating system
Controls all drives attached to it	Performance is dependent on CPU workload
Array(s) appear to the host operating system as a regular disk drive	Does not support all RAID levels
Provided with administrative software	—

SYLLABUS ADDITION: Textbook completion: hardware RAID can be **external** (a separate controller/enclosure with its own processor and cache, preferred for large systems) or **internal** (a RAID card in the host). Software RAID is cheaper but uses host CPU and memory.

2.3 RAID Techniques (Building Blocks)

SYLLABUS ADDITION: The PPT uses these terms without defining them together; they are the basis of all RAID levels.

Technique	Meaning	Gives
Striping	Data is spread across multiple disks. A strip is a group of contiguous blocks written to one disk; a stripe is the set of aligned strips across all disks. Stripe size = strip size × number of data disks.	Performance (parallel access)
Mirroring	Same data is written to two disks (two copies).	Availability / fault tolerance
Parity	An extra calculated value (XOR of data) stored so that lost data can be rebuilt.	Fault tolerance at lower cost than mirroring

How Parity Works

RAID redundancy: parity. Parity is calculated from data and stored on a parity disk. Example from the PPT (sum used to illustrate the idea):

```
Parity calculation: 4 + 6 + 1 + 7 = 18
If one value is lost: 4 + 6 + ? + 7 = 18
? = 18 - 4 - 6 - 7
? = 1
```

In real RAID, parity is the **XOR ($\oplus$)** of the data. Truth table for XOR (same as parity):

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

Recovering from a disk failure

A small RAID level 3 array with data disks **D0** and **D1** and parity disk **P** can tolerate the failure of either D0 or D1. Since $P = D0 \oplus D1$, we get $D1 = D0 \oplus P$ and $D0 = D1 \oplus P$.

D0	D1	$P = D0 \oplus D1$
0	0	0
0	1	1
1	0	1
1	1	0

MINI SUMMARY: RAID = redundant array of inexpensive/independent disks. More disks → lower array MTBF, so RAID adds redundancy. Components: controller + physical disks + logical array. Techniques: striping, mirroring, parity (XOR).

3. RAID Levels

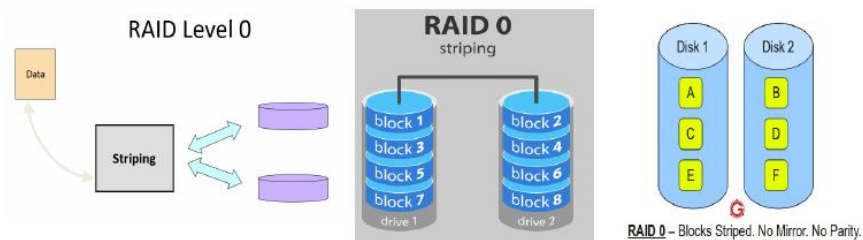
Level	Description
RAID-0	Striped array with no fault tolerance
RAID-1	Disk mirroring
Nested RAID	(i.e., 1+0, 0+1, etc.)
RAID-3	Parallel access array with dedicated parity disk
RAID-4	Striped array with independent disks and a dedicated parity disk
RAID-5	Striped array with independent disks and distributed parity
RAID-6	Striped array with independent disks and dual distributed parity

3.1 RAID 0 – Striping



RAID Level 0

- RAID Level 0 is a non-redundant disk array
- Files are striped across disks, no redundant info
- High read throughput
- Best write throughput (no redundant info to write)
- Any disk failure results in data loss; sometimes a file, sometimes the entire volume



RAID 0: blocks 0,1,2,3 ... distributed across the disks of the RAID set (PPT Slide 53)

- RAID Level 0 is a **non-redundant disk array**. Files are striped across disks, no redundant info.
- Data is distributed across the HDDs in the RAID set. Allows multiple data to be read or written **simultaneously/concurrently**, and therefore improves performance.
- **High read throughput; best write throughput** (no redundant info to write).
- Does **not** provide data protection and availability in the event of disk failures. No replication.
- Any disk failure results in data loss; sometimes a file, sometimes the entire volume.

Advantages	Disadvantages
Simple to implement	If array has n disks , failure rate is n times the failure rate of a single disk

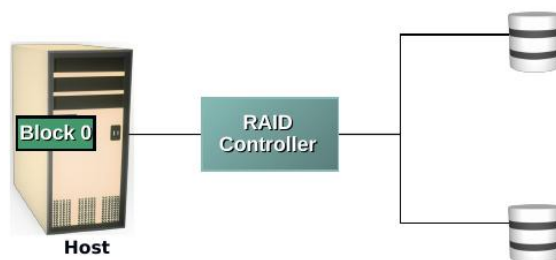
Advantages	Disadvantages
No overhead	No data protection

KEY POINT: Use case: non-critical systems requiring high speed. Mnemonic: RAID **0** = **zero** protection.

3.2 RAID 1 – Mirroring



RAID 1



RAID 1: Host → RAID controller → two mirrored disks (PPT Slide 57)

- Data is stored on **two different HDDs**, yielding two copies of the same data. Provides **availability**.
- In the event of HDD failure, access to data is still available from the surviving HDD.
- When the failed disk is replaced with a new one, data is **automatically copied** from the surviving disk to the new disk. This is done automatically by the RAID controller.
- **Mirroring or shadowing:** two copies of every piece of data → **one logical write = two physical writes; 100 % capacity/space overhead**.
- Files are striped across (half) the disks; data is written to multiple (two) places – data disks and mirror disks. On failure, just use the surviving disk(s). Factor of N (2×) space expansion.

Advantages	Disadvantages
Simple to implement	Requires twice the disk capacity of normal file systems (amount of storage capacity is twice the amount of data stored)
Fault-tolerant	Write is slower than a single disk (every write goes to two disks)

KEY POINT: Mirroring is **NOT** the same as doing backup! (Mirroring copies deletions, corruption and malware too.) Use case: critical systems with moderate performance needs.

3.3 Nested RAID

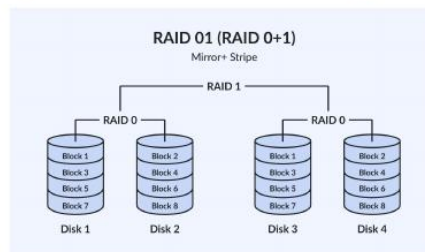
Combines the **performance benefits of RAID 0** with the **redundancy benefit of RAID 1**.

RAID 0+1 – Mirrored Stripe



Nested RAID

- Combines the performance benefits of RAID 0 with the redundancy benefit of RAID 1.
- RAID 0+1 – Mirrored Stripe
 - Data is striped across HDDs, then the entire stripe is mirrored.
 - If one drive fails, the entire stripe is faulted.
 - Rebuild operation requires data to be copied from each disk in the healthy stripe, causing increased load on the surviving disks.



Nested RAID 0+1 (striping and mirroring) (PPT Slide 60)

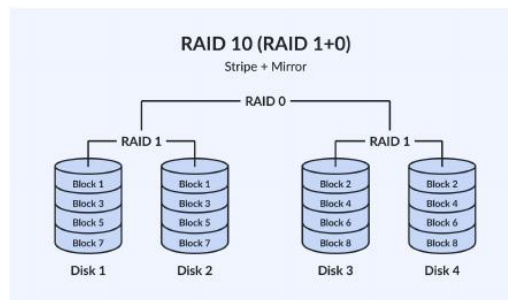
- Data is **striped across HDDs, then the entire stripe is mirrored**.
- If one drive fails, the **entire stripe is faulted**.
- Rebuild operation requires data to be copied from **each disk** in the healthy stripe, causing increased load on the surviving disks.

RAID 1+0 – Striped Mirror

Nested RAID

○ RAID 1+0 – Striped Mirror

- Data is first mirrored, and then both copies are striped across multiple HDDs.
- When a drive fails, data is still accessible from its mirror.
- Rebuild operation only requires data to be copied from the surviving disk into the replacement disk.



Nested RAID 1+0 (mirroring and striping) (PPT Slide 63)

- Data is **first mirrored, and then both copies are striped** across multiple HDDs.
- When a drive fails, data is still accessible from its mirror.
- Rebuild operation only requires data to be copied from the **surviving disk** into the replacement disk.

Parameter	RAID 0+1 (Mirrored Stripe)	RAID 1+0 (Striped Mirror)
Order of operation	Stripe first, then mirror the stripe	Mirror first, then stripe the mirrors
Effect of one drive failure	Entire stripe is faulted	Only that drive is lost; mirror serves data
Rebuild	Copy from each disk of healthy stripe (high load)	Copy only from surviving mirror disk
Preferred?	Less preferred	Preferred (better fault tolerance, faster rebuild)

Both need a **minimum of 4 drives** and have **50 % storage efficiency**. Use case: high-performance systems needing redundancy. (Also nested: **RAID 50 and 60** for enterprise-level storage.)

3.4 RAID 2

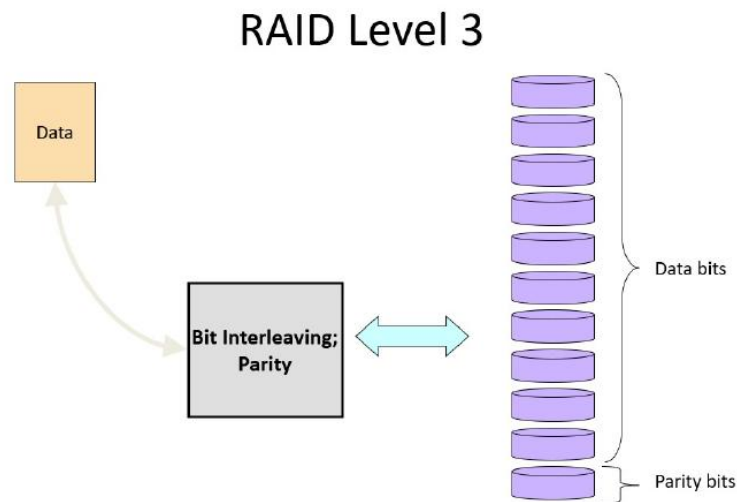
- Instead of duplicating the data blocks we use an **error correction code**.
- Very bad idea because disk drives either work correctly or do not work at all.
- Only possible errors are **omission errors**; we need an **omission correction code**. A **parity bit is enough** to correct a single omission.

3.5 RAID Levels 2, 3 and 4 (common points)

- Use **ECC (error correcting code) or parity disks**. E.g., each byte on the parity disk is a parity function of the corresponding bytes on all the other disks.

- A **read** accesses all the data disks.
- A **write** accesses all the data disks plus the parity disk.
- On disk failure, read the remaining disks plus the parity disk to compute the missing data.

3.6 RAID 3 – Parallel Access with Dedicated Parity



RAID 3: data bits on N disks + parity bits on 1 disk (bit/byte interleaving) (PPT Slide 70)

- Requires **N+1 disk drives**. N drives contain data (1/N of each data block): block $b[k]$ is partitioned into N fragments $b[k,1]$, $b[k,2]$, ... $b[k,N]$.
- Parity drive contains the **exclusive OR** of these N fragments.

FORMULA: $p[k] = b[k,1] \oplus b[k,2] \oplus \dots \oplus b[k,N]$

How RAID 3 works:

1. Assume we have N + 1 disks. Each block is partitioned into N equal chunks (N = 4 in the example).
 2. XOR data chunks to compute the parity chunk.
 3. Each chunk is written into a separate disk.
- **Each read/write involves all disks** in the RAID array. Cannot do two or more reads/writes in parallel. Performance of the array is not better than that of a single disk (for small requests).
 - Stripes data for high performance and uses parity for improved fault tolerance. One drive is dedicated for parity information. If a drive fails, data can be reconstructed using data in the parity drive.
 - Data read/write is done across the **entire stripe** → good bandwidth for **large sequential data access such as video streaming**.

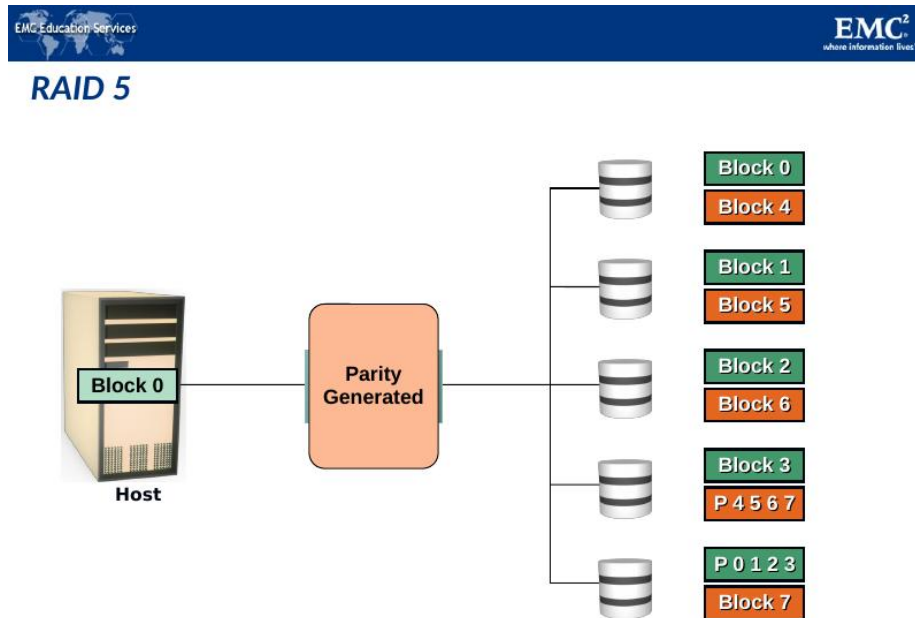
3.7 RAID 4 – Independent Disks with Dedicated Parity

- Requires **N+1 disk drives**. N drives contain data – **individual blocks, not chunks**. Blocks with the same disk address form a **stripe**.
- Parity drive contains the XOR of the N blocks in the stripe.

FORMULA: $p[k] = b[k] \oplus b[k+1] \oplus \dots \oplus b[k+N-1]$

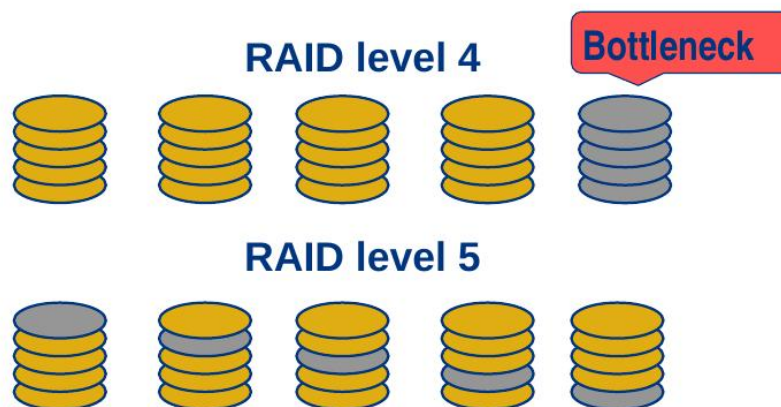
- Parity block now reflects the contents of several blocks. Can now do **parallel reads/writes** (data read/write can be done independently on a single disk).
- **Bottleneck:** the single parity drive is involved in every write, which limits parallelism.

3.8 RAID 5 – Distributed Parity



RAID 5: parity generated and distributed across all disks (PPT Slide 83)

RAID levels 4 and 5



RAID level 4 (dedicated parity disk = bottleneck) vs RAID level 5 (parity spread over all disks) (PPT Slide 80)

- The single parity drive of RAID level 4 is involved in every write → will limit parallelism.

- RAID 5 **distributes the parity blocks among the N+1 drives** – much better.
- RAID 5 is similar to RAID 4, except that the parity is distributed across all disks instead of stored on a dedicated disk. This **overcomes the write bottleneck** on the parity disk.
- Can sustain **one drive failure**. Use case: databases or file servers. Balance of redundancy and performance.

3.9 RAID 6 – Dual Distributed Parity

- RAID 6 is similar to RAID 5, except that it includes a **second parity element** to allow survival in the event of **two disk failures**. (Two check disks; tolerates two disk failures; more complex updates.)
- The probability for two failures increases as the number of drives in the array increases.
- Calculates both **horizontal parity (as in RAID 5) and diagonal parity**.
- Has **more write penalty** than RAID 5. Rebuild operation may take longer than on RAID 5.
- Use case: large storage systems with high fault tolerance.

3.10 Summary Table of RAID Levels

Level	Technique	Fault Tolerance	Key Feature
0	Striping	None	Best performance; no protection
1	Mirroring	1 disk (per pair)	Simple; 100 % overhead
1+0 / 0+1	Stripe + mirror	Multiple (1+0: if no mirrored pair lost)	Performance + redundancy; 4 disks min.
3	Parallel access, dedicated parity	1 disk	Good for large sequential I/O
4	Independent disks, dedicated parity	1 disk	Parity disk bottleneck
5	Independent disks, distributed parity	1 disk	Popular all-round level
6	Dual distributed parity	2 disks	Higher protection; bigger write penalty

MINI SUMMARY: RAID 0 = speed, no safety; RAID 1 = mirror; 1+0 beats 0+1; RAID 3/4 use a dedicated parity disk (bottleneck); RAID 5 distributes parity; RAID 6 uses two parities and survives two failures.

4. RAID Comparison

RAID	Min. Disks	Storage Efficiency %	Cost	Read Performance	Write Performance
0	2	100	Low	Very good for both random and sequential read	Very good
1	2	50	High	Good; better than a single disk	Good; slower than a single disk, as every write must be committed to two disks
3	3	$(n-1) \times 100/n$, n = number of disks	Moderate	Good for random reads and very good for sequential reads	Poor to fair for small random writes; good for large, sequential writes
5	3	$(n-1) \times 100/n$, n = number of disks	Moderate	Very good for random reads; good for sequential reads	Fair for random write (slower due to parity overhead); fair to good for sequential writes
6	4	$(n-2) \times 100/n$, n = number of disks	Moderate but more than RAID 5	Very good for random reads; good for sequential reads	Good for small, random writes (has write penalty)
1+0 and 0+1	4	50	High	Very good	Good

SYLLABUS ADDITION: Usable capacity formulas (from the quiz slides): RAID 0 = sum of all drives; RAID 1 = capacity of one drive (for a pair; $n/2$ for more disks); RAID 5 = $(N-1) \times$ drive capacity; RAID 6 = $(N-2) \times$ drive capacity; RAID 10 = 50 % of total.

5. RAID Impact on Disk Performance

SYLLABUS ADDITION: The PPT only notes the write penalty qualitatively (“slower due to parity overhead”, “more write penalty”). The calculation below is the standard textbook treatment for this syllabus topic.

- Parity RAID (3, 4, 5, 6) must **update parity on every write**, so a single host write becomes multiple disk I/Os. This extra I/O is the **write penalty (WP)**.
- Reads** are not affected by the penalty.

RAID Level	Write Penalty	Reason
0	1	One write, no redundancy
1 / 1+0	2	Each logical write = two physical writes (data + mirror)
5	4	Read old data, read old parity, write new data, write new parity (2 reads + 2 writes)
6	6	As RAID 5 but two parities (3 reads + 3 writes)

FORMULA: Disk load = $(I_p \times R_r) + WP \times (I_p \times R_w)$

I_p = host IOPS; R_r = read fraction; R_w = write fraction; WP = write penalty

Example (illustrative)

Peak workload **5,200 IOPS**, **60 % reads / 40 % writes**, each disk delivers **180 IOPS**.

RAID	Disk load calculation	Disks needed (load / 180)
RAID 1/1+0	$(0.6 \times 5,200) + 2 \times (0.4 \times 5,200) = 3,120 + 4,160 = 7,280$	$7,280 / 180 = 40.4 \rightarrow$ 41 disks
RAID 5	$3,120 + 4 \times 2,080 = 3,120 + 8,320 = 11,440$	$11,440 / 180 = 63.6 \rightarrow$ 64 disks
RAID 6	$3,120 + 6 \times 2,080 = 3,120 + 12,480 = 15,600$	$15,600 / 180 = 86.7 \rightarrow$ 87 disks

- Why this matters:** write-heavy workloads suit RAID 1/1+0; RAID 5 suits read-heavy workloads (databases/file servers with fewer writes). Cache (write-back) helps hide the penalty.
- RAID 0 gives the best performance but no protection; RAID 3 suits large sequential access; RAID 5 handles random reads well.

MINI SUMMARY: RAID improves reads through striping but parity RAID slows writes (penalty: RAID 1 = 2, RAID 5 = 4, RAID 6 = 6). Disk requirement = total disk load / IOPS per disk.

6. Hot Spares

- A **hot spare** is a spare drive kept in the array. If one of the drives within the disk array fails, the hot spare is ready to go, **seamlessly taking over the functions of the failed drive without any action or intervention by the server's administrator**.
- A **cold spare** is a spare part that will **require physical intervention by the user** to engage the device.

Hot Spare	Cold Spare
Already installed and powered in the array	Kept aside; not installed
Takes over automatically; no admin intervention	Needs physical intervention by the user
Rebuild starts immediately → shorter exposure window	Longer time to recover

SYLLABUS ADDITION: When a hot spare is used, the array rebuilds the failed disk's data on it (from mirror or parity). After the failed disk is replaced, data may be copied back and the spare is freed (permanent hot spare vs. temporary).

7. RAID and Flash Drives; Conclusion

7.1 What about flash drives?

- Having no moving parts should mean fewer failures? **Failures still happen.**
- Flash drives **age as they are written to.**
- Irrecoverable read errors occur (at least as frequently as in magnetic disks?).
- Pure Storage uses a proprietary **3D-RAID** organization for their SSD stores.

7.2 Conclusion

- RAID's **original purpose** was to take advantage of Winchester drives that were smaller and cheaper than conventional disk drives – replace a single drive by an array of smaller drives.
- **Current purpose** is to build fault-tolerant file systems that do not need backups.
- Low cost of disk drives made **RAID level 1** attractive for small installations. Otherwise pick **RAID level 6** for higher protection – can tolerate one disk failure **and** irrecoverable read errors.

7.3 Summary

- Disks can cause reliability and performance problems. To mitigate such problems we can adopt “multiple disks” and gain: **more capacity, redundancy, concurrency**.
- To achieve only **redundancy** → mirroring. To achieve only **concurrency** → striping. To achieve **redundancy and concurrency** → RAID (e.g., levels 2, 3, 4 or 5).

8. Intelligent Storage System (ISS)

8.1 Introduction

- Business-critical applications require high levels of **performance, availability, security, and scalability**.
- A hard disk drive is a core element of storage that governs the performance of any storage system.
- Some of the older disk array technologies could not overcome performance constraints due to the limitations of a hard disk and its mechanical components.
- RAID technology made an important contribution to enhancing storage performance and reliability, but hard disk drives even with a RAID implementation could not meet performance requirements of today's applications.
- With advancements in technology, a new breed of storage solutions known as an **intelligent storage system** has evolved. These are the **feature-rich RAID arrays that provide highly optimized I/O processing capabilities**.

8.2 Components of an Intelligent Storage System

An intelligent storage system consists of **four key components: front end, cache, back end, and physical disks**.

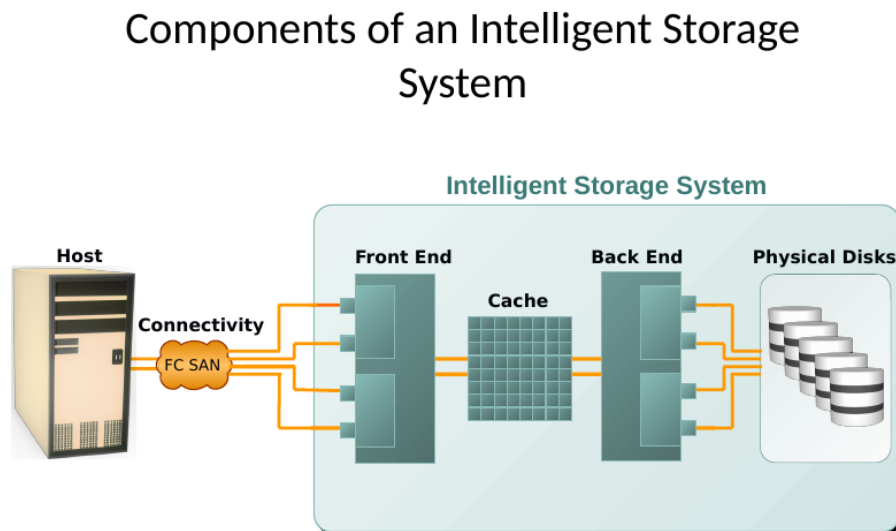


Figure 4-1: Host → Connectivity (FC SAN) → Front End (ports, controllers) → Cache → Back End (controllers, ports) → Physical Disks (PPT Slide 4)

KEY POINT: An I/O request received from the host at the front-end port is processed through cache and the back end, to enable storage and retrieval of data from the physical disk. A read request can be serviced directly from cache if the requested data is found in cache.

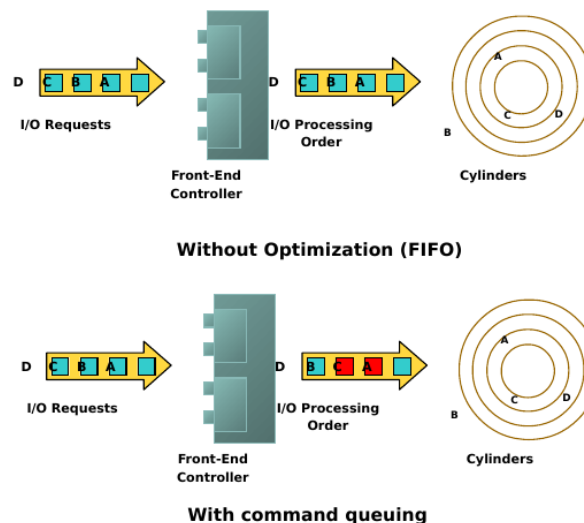
8.3 Front End

- The front end provides the **interface between the storage system and the host**. It consists of two components: **front-end ports** and **front-end controllers**.
- The **front-end ports** enable hosts to connect to the intelligent storage system. Each front-end port has processing logic that executes the appropriate transport protocol, such as **SCSI, Fibre Channel, or iSCSI**, for storage connections. **Redundant ports** are provided on the front end for high availability.
- **Front-end controllers** route data to and from cache via the internal data bus. When cache receives write data, the controller sends an **acknowledgment** message back to the host.
- Controllers optimize I/O processing by using **command queuing algorithms**.

Front-End Command Queuing

Algorithm	Explanation
First In First Out (FIFO)	The default algorithm where commands are executed in the order in which they are received. There is no reordering of requests for optimization; therefore, it is inefficient in terms of performance.
Seek Time Optimization	Commands are executed based on optimizing read/write head movements, which may result in reordering of commands. Without seek time optimization, the commands are executed in the order they are received.
Access Time Optimization	Commands are executed based on the combination of seek time optimization and an analysis of rotational latency for optimal performance.

Front End Command Queuing



Front-end command queuing: without optimization (FIFO) A-B-C-D vs with command queuing A-C-B-D (PPT Slide 8)

Example: the commands are received in the order A, B, C and D. The radial movement required by the head to execute C immediately after A is less than what would be required to execute B. With seek time optimization, the command execution sequence would be **A, C, B and D**.

8.4 Cache

- Cache is an important component that **enhances the I/O performance** in an intelligent storage system.
- Cache is **semiconductor memory** where data is placed **temporarily** to reduce the time required to service I/O requests from the host.
- Cache improves storage system performance by **isolating hosts from the mechanical delays** associated with physical disks, which are the slowest components of an intelligent storage system.
- Accessing data from a physical disk usually takes a few milliseconds because of seek times and rotational latency. If a disk has to be accessed by the host for every I/O operation, requests are queued, which results in a delayed response.

Structure of Cache

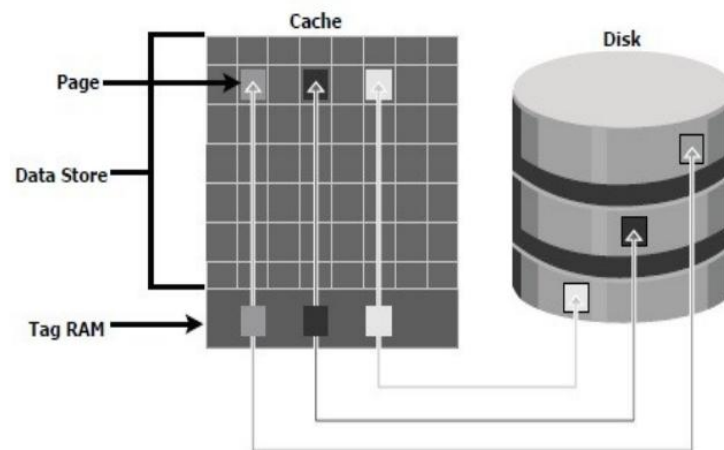


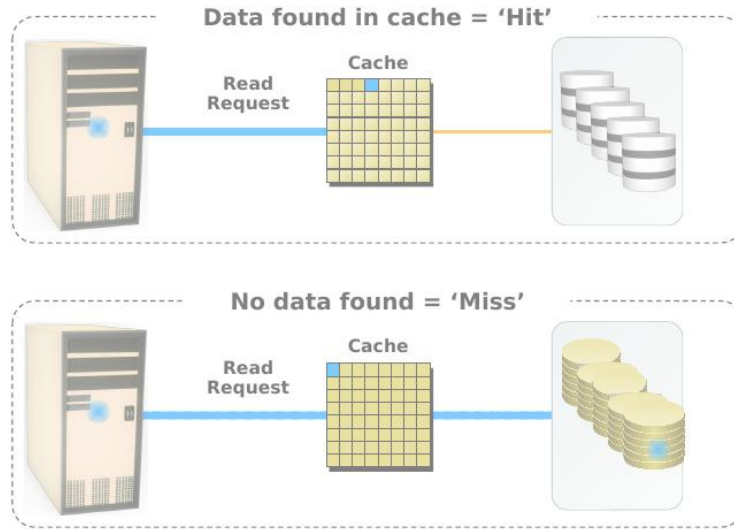
Figure 4-3: Structure of cache

Figure 4-3: Structure of cache – pages, data store, tag RAM, disk (PPT Slide 14)

- Cache is organized into **pages or slots**, which is the smallest unit of cache allocation.
- The size of a cache page is configured according to the application I/O size.
- Cache consists of the **data store** and **tag RAM**. The data store holds the data while tag RAM tracks the location of the data in the data store and in disk.

Read Operation with Cache

Read Operation with Cache: 'Hits' and 'Misses'



Read hit (data found in cache) and read miss (no data found) (PPT Slide 16)

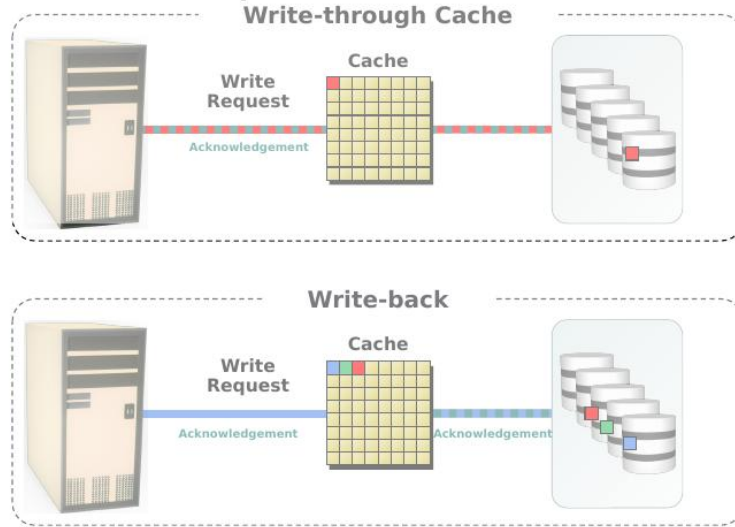
1. When a host issues a read request, the front-end controller accesses the **tag RAM** to determine whether the required data is available in cache.
2. If the requested data is found in the cache, it is called a **read cache hit or read hit** and data is sent directly to the host, without any disk operation. This provides a fast response time to the host (about a millisecond).
3. If the requested data is not found in cache, it is called a **cache miss** and the data must be read from the disk. The back-end controller accesses the appropriate disk and retrieves the requested data. Data is then placed in cache and is finally sent to the host through the front-end controller.
4. **Cache misses increase I/O response time.**

SYLLABUS ADDITION: Textbook completion – read-ahead: **Pre-fetch / read-ahead** loads data into cache before the host asks, helping sequential reads. **Fixed** pre-fetch reads a fixed number of blocks; **variable** pre-fetch adapts to the request size. **Hit ratio** = number of hits / total number of reads.

Write Operation with Cache

- Write operations with cache provide performance advantages over writing directly to disks.
- When an I/O is written to cache and acknowledged, it is completed in far less time (from the host's perspective) than it would take to write directly to disk.
- Sequential writes also offer opportunities for optimization because many smaller writes can be **coalesced for larger transfers** to disk drives with the use of cache.

Write Operation with Cache



Write-through cache vs write-back cache (acknowledgement paths) (PPT Slide 18)

	Write-back Cache	Write-through Cache
Operation	Data is placed in cache and an acknowledgement is sent to the host immediately . Data is committed (destaged) to disk later.	Data is placed in cache and immediately written to disk ; acknowledgement is sent after the disk write completes.
Response time	Faster	Slower
Risk	Data not yet committed is lost if cache fails (volatile)	Little risk of data loss
Cache role	Write buffering	Mainly helps reads

Cache Implementation

- Cache can be implemented as either **dedicated cache or global cache**.
- **Dedicated cache:** separate sets of memory locations are reserved for reads and writes.
- **Global cache:** both reads and writes can use any of the available memory addresses. **Cache management is more efficient** in a global cache implementation, as only one global set of addresses has to be managed.

Cache Management

Cache is a **finite and expensive resource** that needs proper management. Even though intelligent storage systems can be configured with large amounts of cache, when all cache pages are filled, some pages have to be freed up to accommodate new data and avoid performance degradation. Various cache management algorithms proactively maintain a set of free pages and a list of pages that can be potentially freed up whenever required:

Algorithm	Explanation
Least Recently Used (LRU)	Discards least recently used data (data not accessed for a long time is assumed unlikely to be needed). Oldest data is freed for new data.
Most Recently Used (MRU)	Discards most recently used data (it is assumed that recent data may not be required for a while).
Idle flushing	Cache is flushed (de-staged) continuously to disk when I/O load is low, at a modest rate, keeping utilization between LWM and HWM.
High watermark flushing	Flushing is activated when cache utilization reaches the high watermark (HWM) ; dedicated resources are used until utilization falls back to the low watermark (LWM) .
Forced flushing	Occurs when the cache reaches 100 % (very high load); flushing is forced at a high priority, which affects host response time.

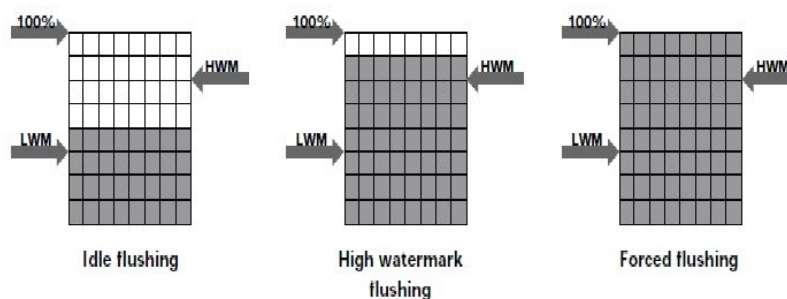


Figure 4-5: Types of flushing

Figure 4-5: Types of flushing – idle, high watermark, forced (LWM, HWM, 100 %) (PPT Slide 21)

- Flushing / de-staging is used to **manage peak I/O request “bursts”**.
- For maximum performance: provide **headroom in write cache** for I/O bursts.

Cache Data Protection

Cache is **volatile memory**, so a power failure or any kind of cache failure will cause the loss of data not yet committed to the disk. This risk can be mitigated using **cache mirroring and cache vaulting**:

- **Cache mirroring**: each write to the cache is held in **two different memory locations on two independent memory cards**. If a cache failure occurs, the write data will still be safe in the mirrored location and can be committed to the disk. Cache coherency: only writes are mirrored (reads can be re-fetched from disk).

- **Cache vaulting:** cache is exposed to the risk of uncommitted data loss due to power failure. In the event of power failure, uncommitted data is **dumped to a dedicated set of drives called vault drives**.

8.5 Back End

- The back end provides an **interface between cache and the physical disks**. It consists of two components: **back-end ports** and **back-end controllers**.
- The back end controls data transfers between cache and the physical disks. From cache, data is sent to the back end and then routed to the destination disk. Physical disks are connected to ports on the back end.
- The back-end controller communicates with the disks when performing reads and writes and also provides additional, but limited, temporary data storage.
- The algorithms implemented on back-end controllers provide **error detection and correction, along with RAID functionality**.

8.6 Physical Disk

- A physical disk **stores data persistently**. Disks are connected to the back end with either **SCSI or a Fibre Channel interface**.
- An intelligent storage system enables the use of a **mixture of SCSI or Fibre Channel drives and IDE/ATA drives**.

MINI SUMMARY: ISS = Front End (ports + controllers, command queuing) + Cache (data store + tag RAM; hit/miss; write-back/through; LRU/MRU; flushing; mirroring/vaulting) + Back End (ports + controllers, RAID, error correction) + Physical Disks.

9. Logical Unit Number (LUN) and LUN Masking

9.1 LUN

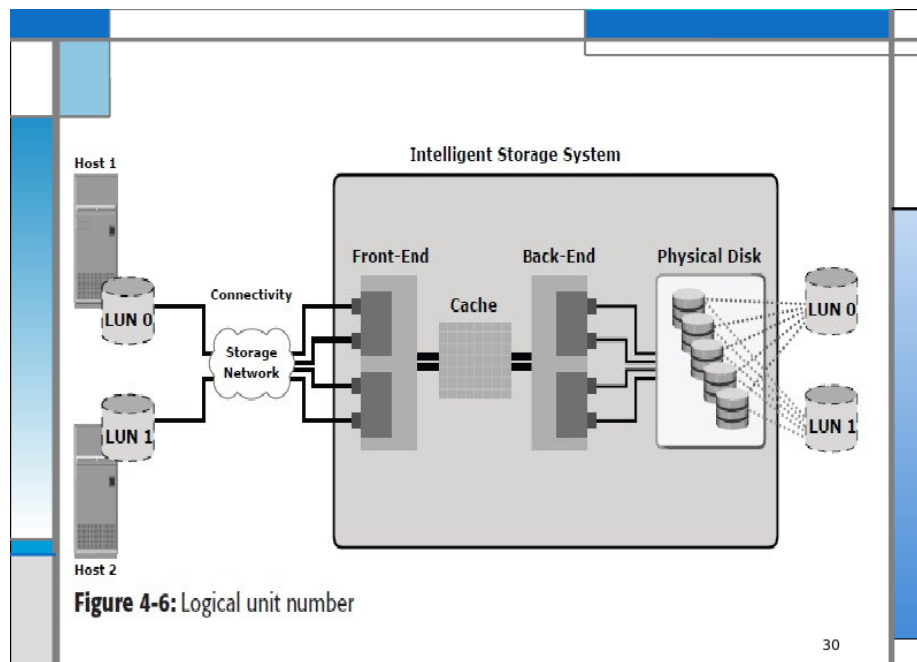


Figure 4-6: Logical unit number – Host 1/Host 2 → storage network → ISS → LUN 0, LUN 1 on physical disks (PPT Slide 30)

- Physical drives or groups of RAID protected drives can be logically split into volumes known as **logical volumes**, commonly referred to as **Logical Unit Numbers (LUNs)**.
- The use of LUNs **improves disk utilization**.
- **Example:** without the use of LUNs, a host requiring only 200 GB could be allocated an entire 1 TB physical disk. Using LUNs, only the required 200 GB would be allocated to the host, allowing the remaining **800 GB** to be allocated to other hosts.

9.2 LUN Masking

DEFINITION: LUN masking is a process that provides **data access control** by defining which LUNs a host can access.

- LUN masking function is typically implemented at the **front-end controller**.
- This ensures that volume access by servers is controlled appropriately, preventing **unauthorized or accidental use** in a distributed environment.
- **Example:** a storage array with two LUNs that store data of the sales and finance departments. Without LUN masking, both departments can easily see and modify each other's data, posing a high risk to data integrity and security. With LUN masking, LUNs are accessible only to the designated hosts.

10. Intelligent Storage Array

Intelligent storage systems generally fall into one of two categories: **High-end storage systems** and **Midrange storage systems**.

10.1 High-end Storage Systems (Active-Active Arrays)

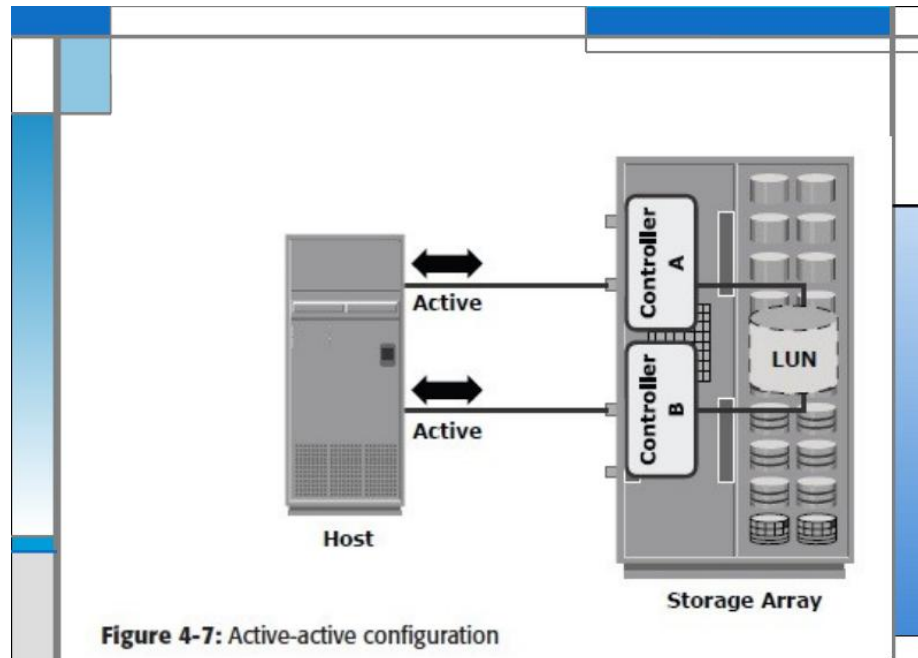


Figure 4-7: Active-active configuration – host connected to both controller A and controller B (both active) to the same LUN (PPT Slide 34)

- High-end storage systems, referred to as **active-active arrays**, are generally aimed at **large enterprises** for centralizing corporate data.
- These arrays are designed with a **large number of controllers and cache memory**.
- An active-active array implies that the host can perform I/Os to its LUNs **across any of the available paths**.

Capabilities provided to address enterprise storage needs:

1. Large storage capacity
2. Large amounts of cache to service host I/Os optimally
3. Fault tolerance architecture to improve data availability
4. Connectivity to mainframe computers and open systems hosts
5. Availability of multiple front-end ports and interface protocols to serve a large number of hosts
6. Availability of multiple back-end Fibre Channel or SCSI RAID controllers to manage disk processing
7. Scalability to support increased connectivity, performance, and storage capacity requirements
8. Ability to handle large amounts of concurrent I/Os from a number of servers and applications
9. Support for array-based local and remote replication

10.2 Midrange Storage Systems (Active-Passive Arrays)

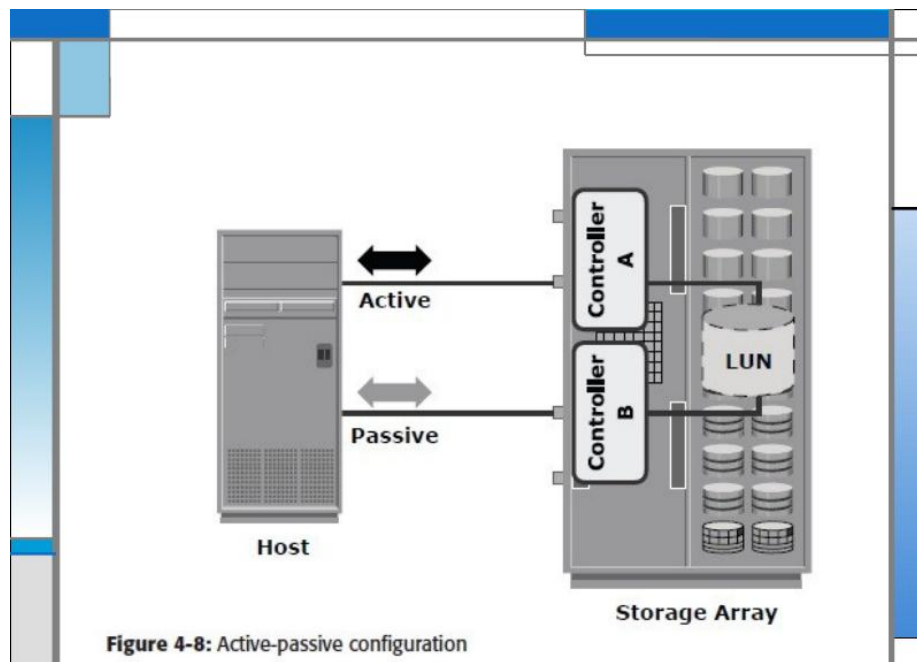


Figure 4-8: Active-passive configuration – path to controller A active, path to controller B passive (PPT Slide 38)

- Midrange storage systems are also referred to as **active-passive arrays** and are best suited for **small- and medium-sized enterprises**.
- In an active-passive array, a host can perform I/Os to a LUN **only through the paths to the owning controller of that LUN**. These paths are called **active paths**. The other paths are **passive** with respect to this LUN.
- As shown in Figure 4-8, the host can perform reads or writes to the LUN only through the path to controller A, as controller A is the owner of that LUN. The path to controller B remains passive and no I/O activity is performed through this path.

Parameter	High-end (Active-Active)	Midrange (Active-Passive)
Target users	Large enterprises	Small and medium-sized enterprises
I/O paths	Host can do I/O to a LUN through any available path	Host can do I/O to a LUN only via the owning controller's (active) paths ; others are passive
Controllers and cache	Large number	Fewer (typically two controllers)
Capacity / scalability	Very high	Moderate
Connectivity	Mainframe and open systems	Typically open systems
Cost	High	Lower

11. EMC Symmetrix Storage Array (Example of High-end Array)

The EMC Symmetrix establishes the highest standards for performance and capacity for an enterprise information storage solution and is recognized as the industry's most trusted storage platform. The **EMC Symmetrix DMX-4** uses the **Direct Matrix Architecture** and incorporates a **fault tolerant design**.

11.1 Features

- Incrementally scalable up to **2,400 disks**
- Supports **Flash-based solid-state drives**
- Dynamic global cache memory (**16 GB–512 GB**)
- Advanced processing power (up to **130 PowerPC**)
- Large number of concurrent data paths available (**32–128 data paths**) for I/O processing
- High data processing bandwidth (up to **128 GB/s**)
- Data protection with **RAID 1, 1+0 (also known as 10 for mainframe), 5, and 6**
- Storage-based local and remote replication for business continuity through **TimeFinder** and **SRDF** software

11.2 Component Overview

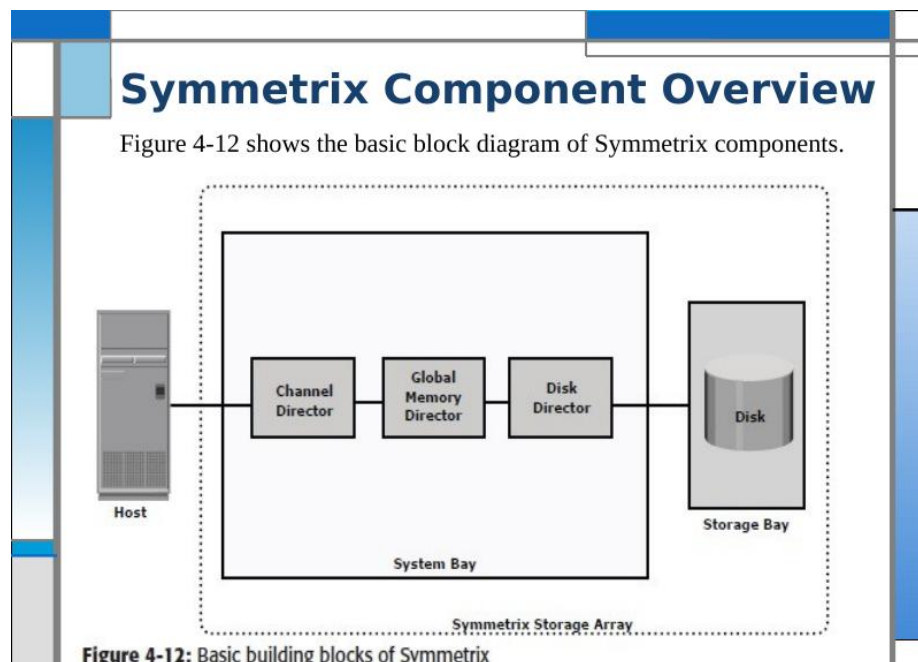


Figure 4-12: Basic building blocks of Symmetrix

Figure 4-12: Basic building blocks of Symmetrix – Host → Channel Director → Global Memory Director → Disk Director → Storage Bay (disks) (PPT Slide 41)

- The Symmetrix **system bay** consists of **front-end controllers (called Channel Directors)** for host connectivity, large amounts of **global cache (called Global Memory Director [GMD])**, and **back-end controllers (called Disk Directors)** for disk connectivity.
- The **storage bay** is a disk enclosure that can house **240 drives** in a cabinet.

11.3 Direct Matrix Architecture

Symmetrix uses the Direct Matrix Architecture consisting of **dedicated paths for data transfer between the front end, global memory, and the back end**. Key components of Symmetrix DMX are:

- Front end
- Back end
- Global Memory
- XCM
- Symmetrix Enginuity (the operating environment/software of the array)

MINI SUMMARY: Symmetrix DMX = high-end active-active array: Channel Directors (front end), Global Memory (cache), Disk Directors (back end), Direct Matrix dedicated paths, up to 2,400 disks, TimeFinder/SRDF replication.

12. Object-Based Storage and Unified Storage Platform

SYLLABUS ADDITION: These two topics belong to the Intelligent Storage Array part of the syllabus but are **not in the uploaded PPT**. They are written from the standard textbook (EMC ISM) so the syllabus is fully covered. Check against your class notes.

12.1 Why Object-Based Storage?

- Data is growing rapidly, and most of the new data is **unstructured** (images, videos, e-mails, documents, web content).
- Traditional **file-based (NAS)** storage uses a **hierarchical file system** (directories and folders). With billions of files, the hierarchy, file-system metadata and locking become a bottleneck for scalability and performance.
- **Block-based (SAN)** storage gives high performance but has no knowledge of the data and no rich metadata, so it is not suited to managing huge volumes of unstructured data.
- **Object-based storage** was developed to overcome these limits: it stores data as **objects in a flat address space**, with rich metadata, and scales to billions of objects across geographies.

12.2 Object-Based Storage Device (OSD)

DEFINITION: Object-based storage is a way of storing file data in the form of **objects**, based on its **content and other attributes**, rather than the name and location of the file.

An **object** is a container of data (user data) and its associated metadata. An object has three parts:

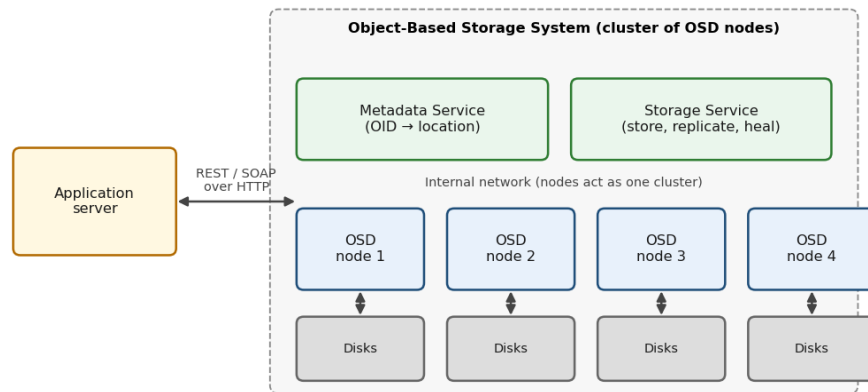
Part	Description
Object ID (OID)	A unique identifier for the object, generated from its content (e.g., using a hash/checksum algorithm such as MD5 or SHA), so identical content gets the same ID. It lets the application find the object in the flat address space without knowing its physical location.
Data (user data)	The actual content: file, image, video, document, etc.
Metadata	Describes the object: system metadata (size, creation time, location, replica count) and custom/user-defined metadata (owner, retention period, content type, business tags). Metadata is stored with the object and drives policies (retention, replication, access).

KEY POINT: Object = **OID + data + metadata**. Flat namespace: objects are accessed by OID, not by folder path. Content-based OID also gives **content addressing** (used in CAS – Content Addressable Storage – for fixed content such as archives).

Components of Object-Based Storage

- **Nodes (OSD nodes):** the storage units that hold objects and run the object storage software. Each node has **controller, CPU, memory and disks**.

- **Internal network:** connects the nodes (e.g., Ethernet/IP) so they behave as one **cluster**; used for replication, data distribution and recovery.
- **Storage:** the disks in nodes that physically store objects.
- **Metadata service (MDS):** maintains the **mapping of OID → physical location** of objects (and object metadata), and handles object lookup.
- **Storage service:** manages the **actual storage of objects** on the disks (placement, replication, protection, healing).
- **Application server / clients:** access the system through **REST/SOAP web-service APIs over HTTP** (not through mounting a volume or NFS/CIFS).



Object-based storage architecture: Application server → (REST/SOAP over HTTP) → Metadata service + Storage service on OSD nodes → Disks, with an internal network connecting the nodes (illustrative diagram – not from PPT)

Object Storage and Retrieval (Operation)

Storing an object:

1. The application server creates an object (data + metadata) and sends it to the OSD using the web-service API (e.g., PUT).
2. The OSD node receives the object, **generates the OID** (from the content), and stores the object.
3. The object is **replicated** (or erasure-coded) to other nodes according to the protection policy.
4. The OSD returns the **OID** to the application server as a receipt.
5. The application server stores the OID in its own database/index to reference the object later.

Retrieving an object:

1. The application server sends a request (GET) with the **OID**.
2. The metadata service looks up the OID and finds the node(s) holding the object.
3. The object is returned to the application; if a node has failed, a replica is served.

Key Features and Benefits

Feature	Explanation
Scalability	Add nodes to scale capacity and performance to petabytes and billions of objects; flat namespace removes hierarchy limits.
Flat address space	All objects in one flat namespace identified by OID; no directory tree.

Feature	Explanation
Rich, custom metadata	Policies (retention, replication, tiering) can be applied per object based on its metadata.
Data protection and self-healing	Replication or erasure coding across nodes; automatic detection and repair of failures, with no RAID needed within a node.
Geographic distribution	Objects can be replicated across sites, giving high availability and disaster recovery.
Multi-tenancy and security	Multiple applications/users share the system with isolation, access control and encryption.
Platform independence / REST access	Accessed over HTTP with REST/SOAP APIs; works well with cloud and web applications.
Low cost	Built from commodity hardware nodes.

Typical use cases: cloud storage and web 2.0 content (photos, videos), **archiving / long-term retention**, content depots, backup and big-data repositories, medical images and e-mail archives.

Limitations: not suited to **transactional, frequently modified data** (databases) – objects are usually written once and read many times; higher access latency than block storage.

Comparison: Block vs File vs Object Storage

Parameter	Block (SAN)	File (NAS)	Object (OSD)
Data unit	Fixed-size block	File in a hierarchical file system	Object (data + metadata + OID)
Address	LUN + block address	Path / file name	Object ID in a flat namespace
Access protocol	FC, iSCSI, FCoE	NFS, CIFS (SMB)	REST / SOAP over HTTP
Metadata	Minimal	Limited (file attributes)	Rich and customizable
Scalability	Limited by array / LUN	Limited by file-system hierarchy	Massive (billions of objects)
Best for	Databases, transactional, low latency	Shared files, home directories	Unstructured data, archives, cloud

12.3 Unified Storage Platform

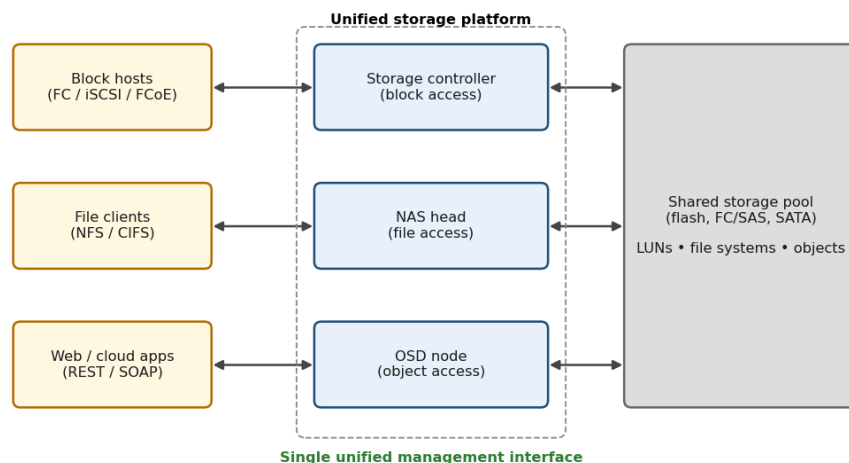
DEFINITION: Unified storage (or unified storage platform) is a **single, integrated storage architecture** that supports **block-level, file-level and object-level access** to storage, managed from **one interface**.

- Earlier, organizations ran **separate** systems: SAN arrays for block data, NAS devices for file data, and OSD systems for object data. This created **silos**, under-utilized capacity and complex management.

- A unified storage platform **consolidates** these into one system that serves **SAN, NAS and object** workloads from a **common pool of storage**.

Components of a Unified Storage Platform

Component	Description
Storage controller	Provides block-level access to hosts over FC, iSCSI and FCoE and manages the back-end disks (RAID, cache). Typically two controllers (active-active or active-passive) for high availability.
NAS head	A dedicated hardware/software file-serving module (CPU, memory, NICs) that provides file-level access using NFS and CIFS over IP. It creates and manages file systems on LUNs provided by the storage controller.
OSD node	Provides object-level access (REST/SOAP APIs) by storing objects as files/blocks in the same storage pool.
Storage (disk pool)	The shared disks (flash, FC/SAS, SATA) from which block LUNs, file systems and objects are all provisioned.
Unified management software	A single GUI/CLI for provisioning, monitoring, replication and reporting of block, file and object storage.



Unified storage architecture: FC/iSCSI/FCoE hosts → storage controller; NFS/CIFS clients → NAS head; REST/web clients → OSD node; all backed by one shared storage pool with unified management (illustrative diagram – not from PPT)

How Data Access Works

- **Block access:** hosts connect through FC/iSCSI/FCoE to the storage controller and use **LUNs** directly.
- **File access:** clients connect over the LAN using NFS/CIFS to the **NAS head**. The NAS head uses LUNs from the storage controller as the base on which it builds file systems and shares them.
- **Object access:** web/cloud applications send REST/SOAP requests to the **OSD node**, which stores objects in the same pool.

Benefits of Unified Storage

1. **Single platform and single management interface** – simpler administration, less training.
2. **Better utilization** of capacity – one shared pool instead of separate silos with stranded free space.

3. **Lower cost** (capex and opex) – less hardware, power, cooling and floor space.
4. **Flexibility** – provision block, file or object storage from the same system as needs change.
5. **Consolidated data protection** – common replication, snapshot and backup policies.
6. **Scalability** – capacity and performance can be grown for all data types together.

Limitations: the shared controller resources are used by all protocols, so heavy file and block workloads can compete; a very large specialized workload may still be better served by a dedicated system.

Unified vs Traditional Storage

Parameter	Traditional (separate systems)	Unified storage
Access types	One type per system (SAN or NAS or OSD)	Block + file + object in one
Management	Separate tools per system	Single management interface
Capacity utilization	Low (silos)	High (shared pool)
Cost	Higher	Lower
Provisioning	Rigid, per system	Flexible, from common pool

MINI SUMMARY: Object-based storage stores data as objects (OID + data + metadata) in a flat namespace, accessed by REST/HTTP; scales to billions of objects and is ideal for unstructured, write-once-read-many data. Unified storage combines block (SAN), file (NAS) and object access on one platform with a single management interface and shared storage pool.

13. Benefits of an Intelligent Storage System

An intelligent storage system provides the following benefits to an organization:

1. Increased capacity
2. Improved performance
3. Easier storage management
4. Improved data availability
5. Improved scalability and flexibility
6. Improved business continuity
7. Improved security and access control
 - An intelligent storage system is now an integral part of every mission-critical data center.
 - Although a high-end intelligent storage system addresses information storage requirements, it poses a challenge for administrators to **share information easily and securely across the enterprise**.
 - **Storage networking** is a flexible information-centric strategy that extends the reach of intelligent storage systems throughout an enterprise. It provides a common way to manage, share, and protect information (detailed in the next unit).

14. Solved Problems and Review Questions

14.1 RAID Capacity (Review Question)

Consider an array consisting of **four 750 GB disks**. What is the storage capacity if organized as RAID 0, RAID 1, RAID 5?

RAID Level	Calculation	Capacity
RAID 0	$4 \times 750 \text{ GB}$	3 TB
RAID 1	$(4 \times 750 \text{ GB}) / 2$ (mirrored pairs)	1.5 TB
RAID 5	$(4 - 1) \times 750 \text{ GB}$	2.25 TB

14.2 Minimum Drives and Usable Capacity (Quiz)

RAID	Minimum Drives	Usable Capacity
RAID 0	2	Sum of all drives
RAID 1	2	Capacity of one drive
RAID 5	3	$(N-1) \times \text{drive capacity}$ (N = number of drives)
RAID 6	4	$(N-2) \times \text{drive capacity}$
RAID 10	4	50 % of total drive capacity

14.3 Problems

Problem 1 (and 3, repeated in the PPT). A RAID 10 array has 8 disks of 1 TB each. What is the usable storage capacity and fault tolerance?

- Usable storage = half the total disk size = $(8 / 2) \times 1 \text{ TB} = \mathbf{4 \text{ TB}}$.
- Fault tolerance: can tolerate multiple disk failures as long as **no mirrored pair is lost**.

Problem 2. A RAID 6 array consists of 6 disks, each 10 TB. What is the total usable storage, and how many disk failures can it tolerate?

- Usable storage = $(\text{number of disks} - 2) \times \text{disk size} = (6 - 2) \times 10 \text{ TB} = \mathbf{40 \text{ TB}}$.
- Tolerates up to **2 disk failures**.

Problem 4. Explain why RAID is not a substitute for backups.

- RAID protects against **hardware failures** but not against **data corruption, accidental deletion, or malware**. Backups provide an **independent data copy** for recovery.

Problem 5. Calculate the probability of failure in a RAID 6 array with 5 disks, each with a 2 % failure rate over a year.

- PPT solution: probability of a single disk failing = 0.02. Probability of two simultaneous failures $\approx 0.02 \times 0.02 = 0.0004$ or **0.04 %** (for a given pair of disks).

SYLLABUS ADDITION: CHECK: RAID 6 survives two failures, so the array fails only when **three or more** disks fail. Exact: $P(\geq 3 \text{ of } 5) = C(5,3)(0.02)^3(0.98)^2 + C(5,4)(0.02)^4(0.98) + (0.02)^5 \approx 7.68 \times 10^{-5} + 7.8 \times 10^{-7}$

$+ 3 \times 10^{-9} \approx 7.8 \times 10^{-5}$ ($\approx 0.0078\%$). Write the PPT answer if the exam follows the PPT, and mention this if asked for the exact failure probability.

Problem 6 – Rebuilding a RAID 5 array. 4 disks (A, B, C, D; parity rotates), 4 blocks each. Disk A fails; rebuild it using parity.

Block	Disk A	Disk B	Disk C	Disk D (Parity)
1	5	7	3	P1
2	2	4	6	P2
3	1	8	2	P3
4	9	3	7	P4

- Parity for each block (XOR): $P1 = A1 \oplus B1 \oplus C1$, $P2 = A2 \oplus B2 \oplus C2$, and so on.
- Rebuilding Disk A for Block 1: known $B1 = 7$, $C1 = 3$, $P1 = 1$.

$A1 = P1 \oplus B1 \oplus C1$
 $A1 = 1 \oplus 7 \oplus 3$
 $A1 = 5$

- Repeat this calculation for all blocks to restore Disk A. (Check: $5 \oplus 7 \oplus 3 = 1 \checkmark$.)

15. Quick Revision

15.1 RAID at a Glance

RAID	Use Case (from PPT quiz)	Purpose / Fault tolerance
0 (Striping)	Non-critical systems requiring high speed	High performance; no redundancy – one drive fails, data lost
1 (Mirroring)	Critical systems with moderate performance needs	Redundancy; copies data to multiple drives
5 (Striping with parity)	Databases or file servers	Balance of redundancy and performance; sustains one drive failure
6 (Dual parity)	Large storage systems with high fault tolerance	Enhanced redundancy; sustains two drive failures
10 (1+0)	High-performance systems needing redundancy	Combines mirroring and striping; high cost; min. four drives
50 and 60	Enterprise-level storage	Nested RAID with more performance and redundancy

15.2 Key Points to Remember

- Array MTBF = drive MTBF / number of drives.
- Mirroring ≠ backup. RAID ≠ backup.
- 1+0 is better than 0+1: faster rebuild, only mirror partner is read.
- RAID 4 parity-disk bottleneck → solved by RAID 5 (distributed parity).
- Write penalty: RAID 1 = 2, RAID 5 = 4, RAID 6 = 6.
- Cache: tag RAM + data store; hit ≈ 1 ms; write-back vs write-through; LRU/MRU; idle/HWM/forced flushing; mirroring + vaulting.
- High-end = active-active; midrange = active-passive. LUN masking is done at the front-end controller.
- Object = OID + data + metadata, flat namespace, REST/HTTP access. Unified storage = block + file + object on one platform (storage controller + NAS head + OSD node).

15.3 Likely Exam Questions

1. What is RAID? Why is RAID needed? (MTBF example)
2. Explain hardware and software RAID implementation and the components of a RAID array.
3. Explain striping, mirroring and parity with examples.
4. Explain RAID 0, 1, 3, 4, 5 and 6 with neat diagrams.
5. Differentiate RAID 0+1 and RAID 1+0.
6. Compare RAID levels (min. disks, efficiency, cost, read/write performance).
7. Explain the impact of RAID on disk performance (write penalty). Calculate disks required.
8. What is a hot spare? Differentiate hot spare and cold spare.
9. Explain the components of an intelligent storage system with a diagram.

10. Explain the front end and command queuing algorithms.
11. Explain cache: structure, read hit/miss, write-back vs write-through.
12. Explain cache management algorithms, flushing, and cache data protection.
13. What is a LUN? Explain LUN masking with an example.
14. Differentiate high-end (active-active) and midrange (active-passive) arrays.
15. Explain EMC Symmetrix architecture and features.
16. Explain object-based storage: object structure, components, and store/retrieve operation.
Compare block, file and object storage.
17. Explain the unified storage platform: components, data access and benefits.
18. Numerical: RAID capacity, parity rebuild using XOR.